## Q1. What is printed when this code runs?

```java
class Animal {
    String speak() { return "..."; }
}

class Dog extends Animal {
    @Override
    String speak() { return "Woof!"; }
}

Animal a = new Dog();
System.out.println(a.speak());
```

1) Woof!
2) Compilation Error
3) Runtime Error
4) ….

## Q2. Does this code follow proper encapsulation?

```java
class Account {
    private double balance;

    public Account(double balance) { this.balance = balance; }

    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
}

Account account = new Account(100.0);
account.setBalance(account.getBalance() - 100);
```

R// false. Even when balance is private, the setter method will allow to modify the state without restrictions or rules.

## Q3. Which SOLID principle does this class violate?

```java
class Employee {
    String name;

    double calculateSalary() { /* ... */ }
    void saveToDatabase() { /* ... */ }
    void printReport() { /* ... */ }
}
```

**[Code]**

A) Open/Closed
B) Liskov, substitution
C) Single Responsibility
D) Interface Segregation

## Q4. Which SOLID principle does this function violate?

```java
double calculateTax(String country, double amount) {
    if (country.equals("CO")) return amount * 0.19;
    else if (country.equals("MX")) return amount * 0.16;
    else if (country.equals("US")) return amount * 0.08;
    return 0;
}
```

A) Open/Closed
B) Liskov, substitution
C) Single Responsibility
D) Interface Segregation

R// Introduce a new country will cause code modification; a better strategy could be to store the taxes in a db table which can be update in runtime.

**Q5. Which version follows the Dependency Inversion Principle?** *[SOLID]*

```java
// Version A:
class OrderService {
    MySQLDatabase db = new MySQLDatabase();
}

// Version B:
class OrderService {
    Database db;

    OrderService(Database db) {
        this.db = db;
    }
}
```

R// B. we should always rely on interfaces rather than specific implementations

## Q6. Which SOLID principle does this function violate?

```java
interface AuthService {
    boolean login(String username, String password);
}

class DatabaseAuthService implements AuthService {
    @Override
    public boolean login(String username, String password) {
        // valida contra base de datos
        return true;
    }
}

class GoogleAuthService implements AuthService {
    @Override
    public boolean login(String username, String password) {
        // Google no usa username/password
        throw new UnsupportedOperationException(
            "Google login no usa username/password"
        );
    }

    public boolean loginWithToken(String token) {
        return true;
    }
}
```

R// Liskov Substition. A subclass should not break the parent behaviour for this case it would be better to create another interface TokenAuth